BEACON

Design decisions and system modelling

How the RAG pipeline, the vector database and the services fit together, and why each choice was made rather than its alternative

Companion to Beacon_Project_Guide.pdf v1.0 • decisions settled 8 November 2026 • v1.1

What this document is for	The guide tells you what to build. This tells you how it is modelled and why each option was chosen over its alternative. Read it before writing code.
Status of the decisions	Settled: Qdrant, local embeddings, filter-inside-search, model-per-step, official data sources only. Open items are listed on the last page.
Where the numbers come from	Model IDs and prices verified against platform.claude.com on 7 Nov 2026. Vector-database sizing from published pgvector/Qdrant benchmarks.
What it becomes later	The seed for docs/ARCHITECTURE.md and the six ADRs in guide section 18.5.

Beacon is decision support for clinicians and patients to discuss together. It is not medical advice, and it is built and evaluated only on synthetic patient cases (TREC Clinical Trials topics).

1. The central problem, and the funnel it forces

375,000 clinical trials; one patient note. You cannot put 375,000 trials in a prompt - not for cost, not for context. So the work splits in two, and **that split is what RAG means**: a cheap stage narrows the corpus, then an expensive stage reads only what survived.

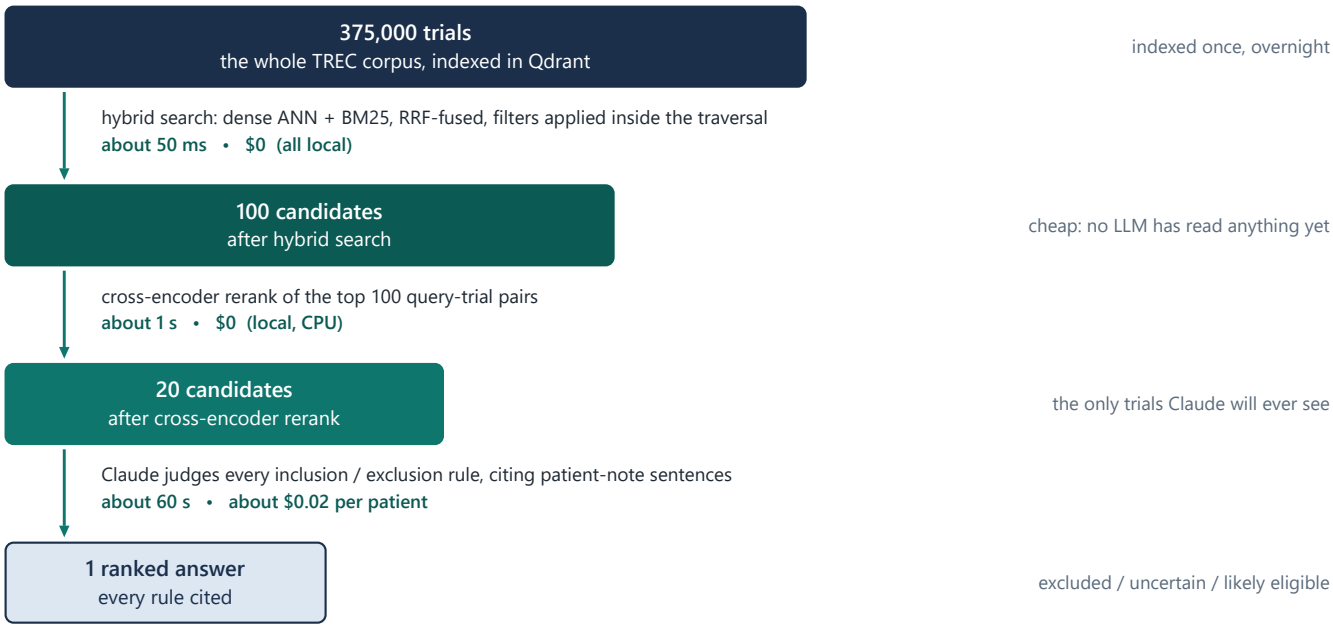


Figure 1. The funnel. Each stage is a different kind of machine: approximate search, then exact pairwise scoring, then language understanding. Cost per stage rises by roughly two orders of magnitude at each step down, which is why the counts fall so sharply.

The consequence that governs everything. Retrieval decides what Claude is *allowed* to see. A good trial dropped at stage one can never be recovered later, however good the model is. That is why **Recall@100** - not accuracy - is the metric the whole system turns on, and why the first real number you produce is the retrieval ablation table.

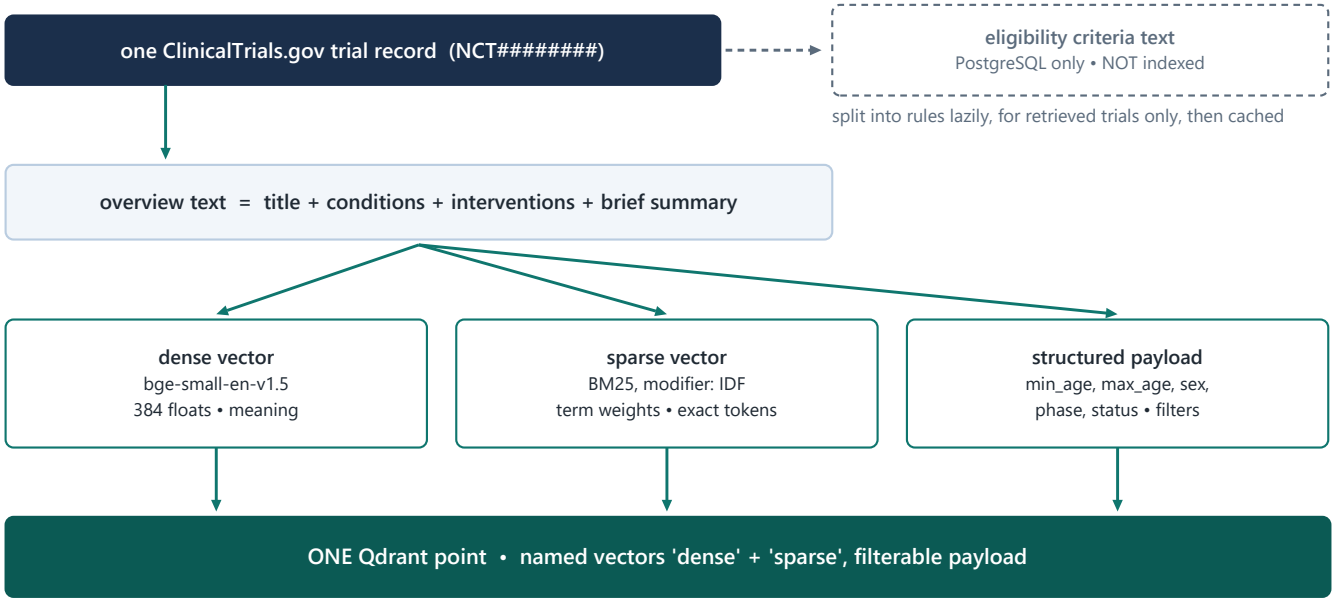
What each stage costs

Stage	Narrows	Mechanism	Time	Money
Retrieve	375,000 to 100	dense ANN + BM25, RRF-fused, filtered inside the index	about 50 ms	$0
Rerank	100 to 20	cross-encoder reads each query-trial pair	about 1 s	$0
Read	20 to a ranked answer	Claude judges every rule, citing note sentences	about 60 s	about $0.02

Retrieval and reranking are free because both models run locally on your CPU. Every dollar Beacon spends is a Claude call - which is exactly what makes the published cost-per-patient figure clean enough to defend.

2. Ingestion: what actually goes into the vector database

This runs once, offline, before any patient is ever matched. For each trial, one short overview text is built and then stored three different ways on a single Qdrant point.



375,000 x 384 dims x 4 bytes = about 576 MB of dense vectors, plus a small sparse index.
Built once on CPU in roughly 30-90 minutes. Re-runnable: idempotent by NCT ID.

Figure 2. One trial becomes one Qdrant point carrying two vectors and a filterable payload. The eligibility criteria - the most important text in the record - is deliberately left out of the index.

Why two vectors and not one

Representation	Built from	What it is good at	Where it fails
Dense vector	overview text to bge-small-en-v1.5, 384 floats	meaning: 'kidney function normal' lands near 'adequate renal function'	blurs rare exact tokens; cannot compare numbers at all
Sparse vector (BM25)	overview text to a term-weight map, heavy on rare terms	exact tokens: EGFR, HbA1c, NCT04179175 - the things that must match literally	no notion of synonymy or paraphrase
Payload	parsed structured fields	exact range and equality tests: age, sex, phase, status	only covers facts that are actually structured in the source

A one-line config that silently costs you accuracy. BM25 needs corpus-level inverse document frequency. Qdrant computes it **only** if the sparse vector field is declared with `modifier: IDF`. Omit it and term weights carry no document-frequency component, so rare-term matching - the entire reason BM25 is here - degrades. No error is raised. You would only notice as a flat ablation table.

What is deliberately not indexed

The eligibility criteria text goes to PostgreSQL as raw text and is split into individual rules only for trials that actually get retrieved, then cached in `trial_criteria` so each trial is parsed once ever. Indexing 375,000 trials times 20-50 rules up front would cost far more than it is worth before anything has measured whether rule-level indexing helps at all. That measurement is a phase-2 ablation, not a day-2 assumption.

2b. Chunking: three different splits, for three different reasons

Standard RAG chops documents into ~500-token overlapping chunks. **Beacon deliberately does not.** There is no single chunking method here - there are three separate splitting decisions, each answering a different question.

Text	Treatment	Why
Trial overview title + conditions + interventions + brief summary	**No chunking.** One overview text becomes one dense vector and one sparse vector	The retrieval unit is a *trial*, not a passage. At 100-400 tokens it fits the embedding window whole
Eligibility criteria long, 20-50 rules per trial	**Structure-aware rule splitting**, never fixed-size. Lazy, and cached in trial_criteria	A rule is the natural semantic unit, and each one is judged individually with its own citation
Patient note	Split into **numbered sentences**	Citation granularity, not retrieval - those numbers are what every judgment points at

The trial side is *compressed, not chunked*: you are not answering 'what does this document say about X', you are asking 'which trials are relevant to this patient', so the natural unit is one trial, one point.

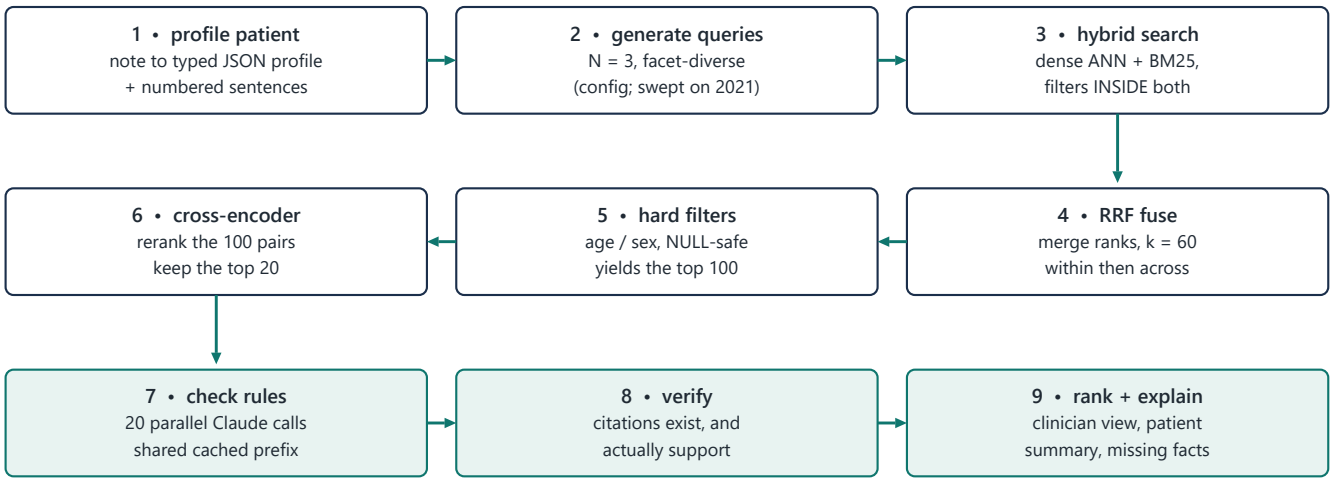
> **Field order is load-bearing, because truncation is silent.** bge-small-en-v1.5 caps at **512 tokens** and simply drops the rest, with no warning. Build the overview with the most discriminative fields first - title, conditions, interventions - and the brief summary last, so what gets cut is the least useful tail. Log how many trials exceed the cap during ingestion; if it is a large fraction, that is itself a finding.

The rule splitter

ClinicalTrials.gov eligibility text is semi-structured: usually an `Inclusion Criteria:` header, bulleted or numbered lines, then `Exclusion Criteria:`. The splitter is rule-based - detect section headers, split on bullets and numbering, join continuation lines - with an LLM fallback only for records too messy to parse. **Never fixed-size character chunking here:** splitting a rule mid-sentence makes its judgment meaningless. This is the component that most needs unit tests against real, messy records.

3. The query path: one patient, nine steps

Sarah, from the guide: 58-year-old woman, non-small-cell lung cancer, EGFR exon 19 deletion, finished chemotherapy in March, creatinine within normal limits.



Steps 1-6 are retrieval: no LLM reads a trial. Steps 7-9 are the read stage, in green.

The filter in step 5 is not a stage after step 3 - it is a constraint evaluated inside both searches.

Step 7 is where 95% of token spend lives, and the only step worth re-running on a bigger model.

Figure 3. The pipeline reads left to right, then wraps. Blue boxes are retrieval and cost nothing; green boxes are the Claude stage.

Step 2 generates three queries, not five to ten - and not to save money. Query count is **not** a cost lever: generation is one call, so ten queries instead of three costs about 140 extra output tokens - roughly $0.00007 per patient, under a cent across a full 125-patient run. Neither real cost scales with it either: the cross-encoder always sees 100 pairs and Claude always checks 20 trials. **Three is chosen for diminishing returns and noise** - queries 1-3 cover the distinct facets, while by query 8 the model is paraphrasing itself and vague restatements pull in loosely-related trials. With only three slots the prompt must enforce **facet diversity**: one query per distinct clinical facet, paraphrases forbidden. N is config, not a constant, and gets swept over {1, 3, 5, 10} on TREC 2021.

The two steps that are easy to get wrong

Step 1 does more than extract fields. It also splits the note into *numbered sentences*. Those numbers are what every later citation points at, so this step is what makes the evidence checkable by code rather than by trust.

Step 7 is one call per trial, and the prompt order is load-bearing:

```
[ stable instructions and label definitions ]    identical across all 20 calls
[ the numbered patient note, sentences 1..N  ]    identical across all 20 calls   <- cache breakpoint
[ this trial's 20-50 inclusion/exclusion rules]    the only part that varies
```

The first two blocks are byte-identical for all 20 trials of one patient, so they form a cacheable prefix: 19 of the 20 calls read them back at a tenth of the input price. Reorder these blocks and the saving silently disappears - the request still succeeds, it just costs more.

Each judgment is schema-constrained, so it cannot come back malformed:

```
class CriterionJudgment(BaseModel):
    criterion_id: str
    kind:  Literal['inclusion', 'exclusion']
    label: Literal['met', 'not_met', 'insufficient_info', 'not_applicable']
    evidence_sentence_ids: list[int]   # must point at real sentences in the note
    explanation: str                   # one or two sentences, for the clinician view
```

Step 8 then checks those citations twice: code confirms every cited sentence ID exists (the invalid citation rate must be 0%), and a cheap second call asks whether the cited sentence actually supports the label. The share that pass is the faithfulness number; its inverse is the measured hallucination rate. **insufficient_info is a success state** - it is how the system abstains instead of guessing, and it feeds the missing-facts list the clinician sees.

4. Where the hard filters go - and why it is not a free choice

Sarah is 58, so trials requiring age 65 or older should not be candidates. The question is *when* that test runs. Two intuitive answers are both wrong, and the second is a named bug.

Option	What happens	Verdict
Filter all 375,000 first, then search the survivors	The vector index is bypassed; you are scanning rows to build a candidate set before searching it.	**Wrong - throws away the index**
Search for the top 20, then filter the results	With ef_search = 40 and a filter matching 10% of rows, you average 4 survivors out of 40. You asked for 20 candidates and received 4.	**Wrong - this is the overfiltering bug**
Filter *inside* the index traversal	The graph walk only visits points satisfying the filter, so the top 20 means 20 age-eligible trials.	**Correct**

```
query  ->  [ filtered dense ANN ]  -+
       ->  [ filtered BM25       ]  -+->  RRF  ->  top 100  ->  cross-encoder  ->  top 20  ->  Claude
                  ^
           the filter lives HERE, inside both searches
```

Qdrant does this natively through payload indexes and filterable HNSW. For the record, pgvector could not do it until 0.8, which added iterative index scans (`hnsw.iterative_scan = 'relaxed_order'`) to keep pulling candidates until k are genuinely satisfied - worth knowing, because it is the kind of detail an interviewer uses to check whether you understood the problem or copied a tutorial.

Filtering before the reranker also matters for cost: the cross-encoder is the expensive local stage, and spending 100 forward passes scoring trials that an integer comparison already ruled out is pure waste.

Two rules that follow

- **Only cheap, exact, structured facts may be filters** - age, sex, and later status = recruiting. 'No brain metastases' and 'ECOG 0-1' are *not* filters; they are the rule checker's job. Promoting a soft criterion to a hard filter is how you silently delete good trials.
- **A filter may exclude only on positive evidence of ineligibility.** ClinicalTrials.gov age fields are frequently absent or unparseable - '18 Years', '6 Months', 'N/A', missing entirely. If min_age is unknown the trial must *survive* the filter and let Claude decide. NULL means 'possibly eligible', never 'exclude'.

Why the second rule is the dangerous direction. A wrongly *included* trial gets caught downstream - the rule checker labels it not_met and the trial is marked excluded, in front of a clinician who can see the reasoning. A wrongly *excluded* trial never appears anywhere, for anyone, with no error logged. This asymmetry is why '+ age/sex filters' is its own ablation row, and why that row is **allowed to come out negative**.

5. Bi-encoder and cross-encoder: the one concept that explains the funnel

Both are neural models that score how well a query matches a trial. The difference is *when* they can do the work, and that single difference dictates the shape of the whole pipeline.

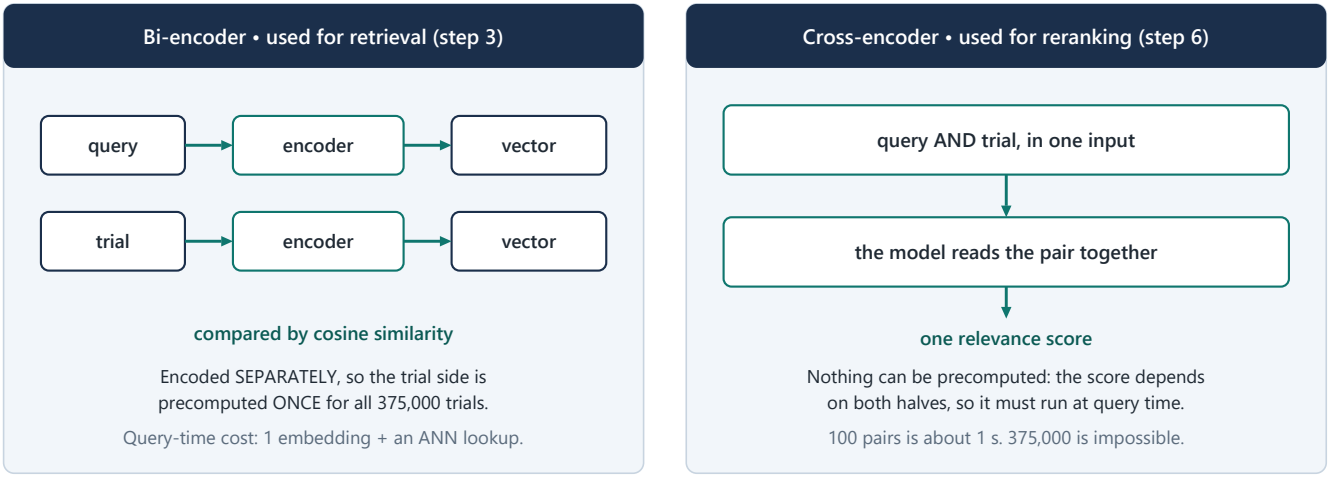


Figure 4. The same task, two architectures. Separating the inputs buys precomputation; combining them buys accuracy. You cannot have both.

	Bi-encoder (retrieval)	Cross-encoder (reranking)
Model here	`BAAI/bge-small-en-v1.5`	`BAAI/bge-reranker-base`
Sees	query and trial separately, never together	query and trial together, in one forward pass
Precomputable	yes - all 375,000 trial vectors, built overnight	no - the score needs both halves at once
Query-time work	one embedding, then an approximate-nearest-neighbour lookup	one forward pass per candidate pair
Throughput	375,000 trials searched in milliseconds	about 100 pairs per second on CPU
Accuracy	approximate - good enough to pick 100 from 375,000	high - good enough to pick the 20 that Claude will read

That asymmetry *is* the funnel: the bi-encoder gets you to 100 because it precomputes, the cross-encoder gets you to 20 because it does not, and Claude judges the 20 because it is the only one that can read eligibility rules. 'Why rerank only the top 100?' appears in the guide's interview list for exactly this reason - and the answer must be a cost/latency curve you measured, not a number you inherited from this document.

6. Embeddings cannot compare numbers

A 24-year-old's note **will** retrieve a trial whose rule is 'age 25 or older', with a high score. Embeddings have no mechanism for numeric comparison: 24 and 25 are just tokens, and cosine similarity has no notion of greater-than. Cross-encoders are somewhat better, because they read both texts together, but they remain unreliable numeric reasoners. **Never depend on either for a threshold.**

This is not a defect, because retrieval does not decide eligibility. Three layers do different jobs, and only one of them decides:

Layer	The question it answers	Reliable with numbers?	Decides eligibility?
Embeddings + BM25 + reranker	Is this trial *about* the right disease?	**No** - and does not need to be	No
Hard payload filters	Is age within [min_age, max_age]? Does sex match?	**Yes** - an integer comparison, not a similarity	Partly - it can only exclude
LLM rule checker	Does this patient meet *this specific rule*, and which sentence proves it?	**Yes** - age arrives as a typed int on the profile	**Yes**

So a retrieved trial is a **candidate**, never a suggestion. Any inclusion rule labelled not_met marks the whole trial excluded in the final ranking. The 24-year-old case is therefore stopped twice, independently: by the age range filter in step 5, and by the rule checker in step 7. Numeric correctness comes from a comparison on structured columns - which is precisely why age and sex are extracted into typed fields instead of being left as prose inside the embedded text.

> **The benchmark is built on the same assumption.** TREC grades each trial **eligible (2) / excluded (1) / not relevant (0)** - and 'excluded' scores *above* 'not relevant'. The benchmark is saying that retrieving a right-disease-wrong-eligibility trial is partially correct retrieval. **Retrieval is measured on relevance; eligibility is the LLM's job.** Hold that distinction and the metric choices in guide section 10 stop looking arbitrary.

The restrictions no filter can catch

'No brain metastases', 'ECOG performance status 0-1', 'no prior immunotherapy' - none of these is a structured field, so no filter can test them. They rest entirely on the rule checker, which is why insufficient_info plus the missing-facts list is a designed feature rather than a shortcoming: for Sarah, 'no brain metastases' is not mentioned in her note at all, and the honest output is 'not enough information - ask for a brain MRI result', not a guess in either direction.

7. The services, and the one problem each one solves

Everything in sections 1-6 would run fine as a single Python script. Each service below exists because of one specific thing that breaks when you do that - and each is a defensible interview answer rather than resume decoration.

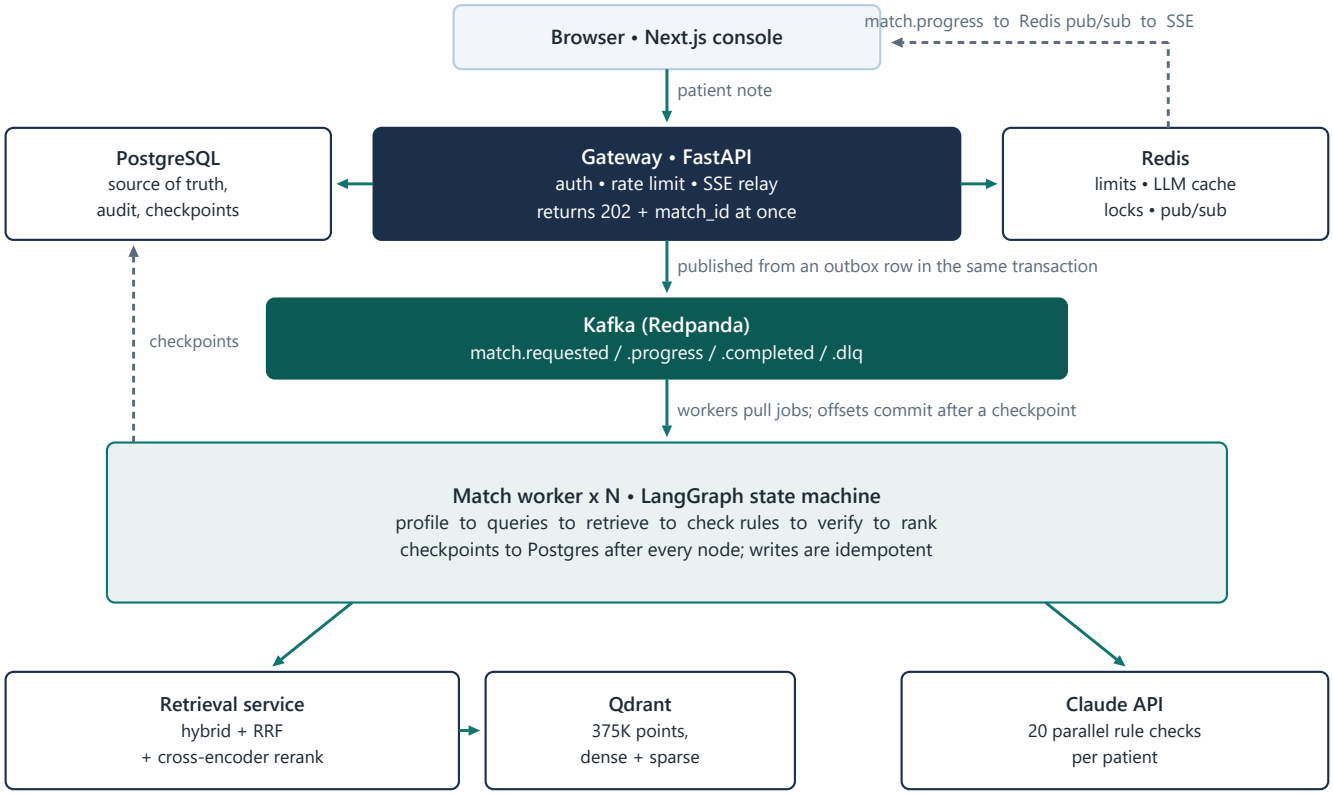


Figure 5. Solid arrows are the request path; dashed arrows are the progress and checkpoint paths.

Why each piece exists

Piece	The problem it solves
Gateway returns 202, not the answer	One patient takes about 90 seconds. An HTTP request held open that long dies to proxy and client timeouts. So: accept the note, return a match_id immediately, and stream progress separately.
Kafka instead of a thread pool	A thread pool (or Spring's @Async) lives inside the process: kill the process and the job is gone. Kafka holds the message until a consumer *commits* it, so a crashed worker's job is redelivered. Replay, consumer-group scaling and backpressure come free. This is the single strongest production signal in the project.
Transactional outbox	'Save to Postgres, then publish to Kafka' is two operations that can half-fail, leaving a request with no job or a job with no request. Writing the row and the pending event in one transaction, then publishing from that table, makes them impossible to desynchronise.
LangGraph + Postgres checkpoints	The six-step pipeline is a state machine. Checkpointing after each node means a worker killed during step 5 resumes at step 5, not step 1. This is what makes the demo's 'kill a worker mid-run' clip actually work.

Piece	The problem it solves
Idempotent writes	Kafka is at-least-once: the same message *will* occasionally arrive twice. Unique constraints plus Redis idempotency keys make a repeat a no-op instead of a duplicate result.
Separate retrieval service	The cross-encoder is CPU-heavy and is called by the workers (and later by the MCP server). Different scaling profile from the gateway, so it scales on its own.
Redis	Rate-limit token buckets as an atomic Lua script, an exact-match LLM response cache, SET NX locks for the ingest job, and pub/sub to fan SSE updates out to whichever gateway replica holds the browser connection.
PostgreSQL	Source of truth: trials, parsed rules, judgments, and the llm_calls audit log that every cost and latency number in the README is computed from.
Phoenix + OpenTelemetry	One trace per request, propagated through Kafka message headers, so you can open a single patient and see all six nodes and all twenty LLM calls with tokens, cost and latency. Without it you cannot answer 'why did that patient cost four cents?'

8. The decisions, each with the option that lost

A decision record is only useful if it names the alternative and why it was rejected. These six become the ADRs in guide section 18.5.

#	Decision	The option that lost, and why
0001	**Qdrant** for vectors ACCEPTED 7 Nov	**pgvector + pg_search.** Not rejected on scale - pgvector is comfortable to about 5M vectors and 375K is small (see section 9). Rejected because Postgres has no real BM25: tsvector uses ts_rank, a different function, so the standard keyword baseline that TREC and TrialGPT results are compared against would be gone. Qdrant also expresses dense + sparse + RRF in one query with IDF handled natively. **Price paid:** a dual write (Postgres, then Qdrant) during ingestion, made safe by being idempotent per NCT ID.
0002	**Kafka (Redpanda)** for jobs	**Redis Streams, or an in-process task queue.** Both lose jobs when the process dies, and neither offers replay or a dead-letter queue. Redpanda is a single Kafka-compatible container, so the operational cost is one service, not a cluster.
0003	**LangGraph for orchestration, raw Anthropic SDK inside each node**	**LangChain chains, or LlamaIndex.** They would write the retrieval and prompting for you - which is exactly the problem, because a project deep-dive interview will ask why a chunk size or a prompt order is what it is. Keeping the SDK direct preserves full control of prompts, caching, token accounting and structured outputs.
0004	**Trial-level indexing** (one vector per trial)	**Rule-level chunks** (one vector per eligibility criterion, prefixed with trial title and condition). Plausibly better and genuinely interesting - but it multiplies the index by 20-50x before anything has measured the benefit. Deferred to a phase-2 ablation, where it is a measured result instead of an assumption.
0005	**Filter inside the index traversal**	**Post-filtering the top k.** Produces the overfiltering bug in section 4: a 10%-selective filter leaves about 4 of 40 results. Also rejected: pre-scanning all 375K, which bypasses the index entirely.
0006	**Haiku 5.5 for development, Sonnet 5.5 for the headline run, Opus 5.5 in the judge role only** LOCKED 8 Nov	**Opus 5.5 for the headline run** - rejected on cost: about $62 batched against Sonnet's $31, for a benchmark number, with no measured evidence the extra capability changes the ranking. **One model everywhere** - rejected twice over: Opus-only kills iteration at about $125 a run, and Haiku-only would have the judge grading output from its own model, which is not a measurement. Opus survives only as the judge, on a ~60-pair calibration sample, for about $1.

9. The vector-database question, answered with numbers

'Do we really need Qdrant - isn't it for scaling?' is the right question to ask, and the honest answer is that **the scale argument does not apply at this size.** Any ADR that claims otherwise is wrong, and an interviewer will say so.

Measure	pgvector	Qdrant	Beacon's position
Comfortable up to	about 5M vectors	tens of millions, distributed	**375,000 - small for both**
Latency at 500K vectors	p50 4.8 ms, p99 9.2 ms	p50 3.1 ms, p99 6.4 ms	**target is p95 under 500 ms - both pass by 50x**
Degrades past	5-10M vectors; index must fit in RAM	not a concern at this scale	**not reached**
Real BM25	**No** - tsvector/ts_rank is a different function; needs the pg_search extension	**Yes** - native sparse vectors with IDF	**the deciding factor**
RRF fusion	hand-written in SQL	server-side, one query via prefetch	Qdrant
Filtering	post-filter until 0.8; iterative index scans since	inside the graph traversal	Qdrant, by default
Age/sex filters	a plain WHERE on the same table	duplicated into the point payload	pgvector is cleaner here
Dual write	**none** - one transaction with the trials table	**yes** - Postgres, then Qdrant	the price of the decision
Containers	5	6	one more service to run

The honest framing for ADR 0001. Choose Qdrant because **hybrid search is the claim this project rests on**, and Qdrant does dense + sparse + RRF in one query with corpus IDF handled for you. Do not choose it because of scale. If the schedule slips, collapsing to pgvector + pg_search - five containers, no dual write - is a legitimate fallback that costs you the server-side-fusion story and nothing else.

The embedding stack: free, local, and that matters

Component	Model	Size	Runs	Cost
Dense embeddings	`BAAI/bge-small-en-v1.5` (384-dim)	about 130 MB	local, CPU, via FastEmbed	**$0**
Sparse / BM25	`Qdrant/bm25`	negligible	local, pure computation	**$0**
Reranker	`BAAI/bge-reranker-base`	about 280 MB	local, CPU	**$0**
Phase 2, biomedical	`ncbi/MedCPT` (768-dim)	about 440 MB	local, GPU helps	**$0**

Anthropic has no embeddings API, and paid alternatives such as Voyage or OpenAI buy nothing here: the corpus is static, so it is embedded *once*, overnight. Paying per token to redo that repeatedly would be strictly worse. This is also load-bearing for the cost story - because retrieval is $0, the entire cost-per-patient figure is Claude calls and nothing else.

Data sources: two, both official, no scraping

Source	What it provides	Used for
TREC Clinical Trials 2021 / 2022 (NIST), loaded with ir_datasets	375,000-trial corpus, 125 synthetic patient topics, expert relevance labels	the benchmark and the answer key
ClinicalTrials.gov API v2 (clinicaltrials.gov/data-api/api)	official JSON REST: current trials, recruiting status, site coordinates	phase-2 live mode only

Both are government-operated. No scraping of third-party trial aggregators: unreliable, terms-hostile, and it would break comparability with TREC and TrialGPT. Claude's web search stays out of the matching loop entirely - every citation must resolve into the indexed corpus, or the faithfulness metric measures nothing. Only the ingestion job touches the network.

A naming clarification worth fixing early. The phase-2 MCP server exposes search_trials and check_eligibility **outward** - they are tools other assistants call in order to use Beacon. It is not a mechanism for pulling trial data in. Those are opposite directions, and conflating them leads to the wrong design.

10. Which model runs each step, and what it costs

All three current models use adaptive thinking, so the only knob per step is `output_config.effort`. Development and every bulk run use Haiku 5.5; the headline number is produced once, on Sonnet 5.5 - locked on 8 Nov 2026.

Pipeline step	Model	Effort	Why this model
`profile_patient`	`claude-haiku-5-5`	medium	Structured extraction from one note, schema-constrained - a small model is genuinely fine
`generate_queries`	`claude-haiku-5-5`	low	Short output, and errors are self-correcting: more queries cover for one weak query
`check_rules` **x20**	`claude-haiku-5-5` in dev, `claude-sonnet-5-5` for the headline run	high	**The accuracy-critical step. About 95% of token spend lives here, and it is the only step worth re-running on a bigger model**
`verify`	`claude-haiku-5-5`	low	A narrow question: does this one sentence support this one label?
`rank_and_explain`	`claude-haiku-5-5`	medium	Summarisation; reading level is graded separately with textstat
LLM judge (evals)	`claude-opus-5-5`	high	**Must be a different, stronger model than the one being judged - otherwise you are grading a model with itself. Calibrate against about 60 of your own hand labels using Cohen's kappa**

Verified prices (platform.claude.com, 7 November 2026)

Model	Model ID	Input / MTok	Output / MTok	Cache read
Opus 5.5	`claude-opus-5-5`	$4.00	$20.00	$0.20 (0.05x)
Sonnet 5.5	`claude-sonnet-5-5`	$2.00	$10.00	$0.10 (0.05x)
Haiku 5.5, prompt under 100K	`claude-haiku-5-5`	$0.10	$0.50	$0.01 (0.1x)

Model	Model ID	Input / MTok	Output / MTok	Cache read
Haiku 5.5, prompt over 100K	`claude-haiku-5-5`	$0.50	$2.50	$0.05

Batch API requests are 50% off both directions, and stack with prompt caching. The minimum cacheable prefix is 512 tokens on all three models - stable instructions plus a numbered patient note clear that easily, but below the minimum caching silently does nothing and returns no error.

What Beacon actually costs

Run	Model	Cost	Role in the project
Development iteration, 10-20 patient subset	Haiku 5.5	**~$0.25-0.50**	**what you actually use while tuning**
Full 125-patient run (2021 + 2022)	Haiku 5.5	about $3	run when a configuration is settled
Headline run, held-out 2022 (50 patients), batched	**Sonnet 5.5**	**~$12.50**	**the number you publish**
Full 125-patient run, batched, both splits	Sonnet 5.5	about $31	optional
LLM judge: calibration + faithfulness sample	Opus 5.5	about $1	independence of the judge

Assumes the guide's figures: 20 trials per patient, about 5,000 input and 1,500 output tokens per trial check. **Iterate on a subset, not the full benchmark** - ten full 125-patient runs is $30, ten 20-patient runs is $5 and tells you the same thing while tuning. Total project spend should land near **$20-40**, comfortably inside the guide's $30-80 envelope and lower than the Opus path would have been. **The bigger model is a final measurement, not a development tool.**

> **A pricing cliff the guide does not mention.** Haiku 5.5 is priced *by prompt length*: above 100,000 input tokens the rates jump 5x, to $0.50 / $2.50. The threshold counts **all** input tokens, cache reads and writes included, and each request is priced on its own. Beacon's per-trial check is about 5,000 tokens, far below - but an innocent-looking refactor to 'batch all 20 trials into one call' would cross it and quintuple the bill with no warning. The LLM wrapper should count tokens before every call, refuse above a configured ceiling, and record the billed tier in the llm_calls audit row.

11. What is deliberately deferred, and what is still open

Deferred on purpose - not forgotten

Deferred	Why, and when it comes back
Rule-level chunking	ADR 0004. Multiplies the index 20-50x. Returns as a phase-2 ablation, so it becomes a measured result rather than an assumption.
MedCPT biomedical embeddings	A plausible accuracy win, but it changes the retrieval baseline. Returns as a phase-2 ablation against bge-small, measured on the same split.
Live ClinicalTrials.gov sync	Phase 2. Turns Beacon from a benchmark system into a live one (recruiting-only, distance to site). Needs the daily trials.updated topic and incremental re-indexing.
MCP server	Phase 2. Cheap to add once the retrieval service exists, and a strong demo - but it earns nothing before the benchmark numbers exist.
Kubernetes, Terraform, Prometheus/Grafana	Docker Compose is sufficient for one laptop, and Phoenix covers the AI-specific observability. None of these improves the headline before 15 November.

Genuinely open - decide with measurements, not opinions

Open question	How it gets settled
Do the age/sex hard filters help or hurt?	The '+ age/sex filters' ablation row. If parsing is weak it can come out negative, and that is a legitimate result to publish rather than hide.
How many candidates should the cross-encoder rerank?	Measure the nDCG@10 and latency curve at 50 / 100 / 200. The guide's 100 is a starting point, not a finding.
Does LLM query generation beat searching with the raw note?	Its own ablation row. Plausible either way - the note contains useful context a short query loses.
Is RRF k = 60 right for this corpus?	Sweep it on TREC 2021 only. Report the chosen value and that you tested alternatives.
Does the verifier's cheap second pass actually catch anything?	Compare verifier verdicts against the hand-labelled sample. If it never disagrees, it is cost with no benefit and should be cut.

The one rule that outranks everything in this document. Every number here is an *estimate* or a *published price*. Nothing in the README, the resume or an interview may cite a performance figure that you did not measure yourself on the held-out 2022 split. Tune on TREC 2021, report on TREC 2022, and never look at 2022 while tuning. An estimate presented as a result is the one mistake that cannot be recovered from.